

Lista de Exercícios de Fixação

1. Explique as diferenças entre SCRUM e Kanban. Em que situações seria mais adequado usar cada metodologia?

- SCRUM: metodologia ágil baseada em ciclos fixos chamados Sprints (1 a 4 semanas), com papéis definidos (Product Owner, Scrum Master, equipe de desenvolvimento), cerimônias regulares (planning, daily, review, retrospectiva) e entregas incrementais a cada Sprint. É mais estruturado e prescritivo.
- Kanban: sistema visual de gestão do fluxo de trabalho sem iterações fixas. Utiliza um quadro com colunas representando os estados das tarefas (A Fazer, Em Progresso, Concluído) e limita o trabalho em andamento (WIP - Work in Progress) para reduzir gargalos. É mais flexível e contínuo.

Quando usar cada um:

- SCRUM: ideal para projetos com escopo definido e planejamento por fases, como o desenvolvimento de um novo produto ou funcionalidade complexa, onde entregas regulares e alinhamento frequente com stakeholders são essenciais.
- Kanban: ideal para equipes de suporte, manutenção ou operações, onde as demandas chegam de forma contínua e imprevisível, como um time de atendimento a bugs ou melhorias incrementais em produção.

2. Descreva as responsabilidades de um Product Owner no SCRUM. Por que esse papel é importante no desenvolvimento de sistemas?

O Product Owner (PO) é o responsável por maximizar o valor do produto desenvolvido pela equipe. Suas principais responsabilidades são:

- Gerenciar o Product Backlog: criar, priorizar e manter atualizada a lista de funcionalidades, melhorias e correções do produto.
- Definir a visão do produto: garantir que toda a equipe compreenda os objetivos de negócio e o valor esperado de cada entrega.
- Ser a ponte entre negócio e tecnologia: traduzir as necessidades dos stakeholders em histórias de usuário claras e priorizadas para a equipe de desenvolvimento.
- Aceitar ou rejeitar entregas: ao final de cada Sprint, o PO avalia se o que foi desenvolvido atende aos critérios de aceitação definidos.

Importância: sem um PO bem definido, a equipe de desenvolvimento pode trabalhar em funcionalidades de baixo valor ou sem alinhamento estratégico, desperdiçando tempo e recursos.

3. Explique como a prática de programação em par do XP pode melhorar a qualidade do código. Dê um exemplo de quando essa prática seria benéfica.

A programação em par (Pair Programming) é uma prática do Extreme Programming (XP) em que dois desenvolvedores trabalham juntos no mesmo computador: um escreve o código (driver) enquanto o outro revisa em tempo real (navigator). Isso melhora a qualidade do código porque:

- Revisão contínua: erros, falhas lógicas e más práticas são identificados imediatamente, antes de serem integrados ao código base.

- Transferência de conhecimento: o conhecimento é compartilhado entre os dois programadores, reduzindo a dependência de uma única pessoa para partes críticas do sistema.
- Código mais limpo: a presença de um revisor em tempo real incentiva naturalmente a escrita de código mais legível e bem estruturado.

Exemplo benéfico: ao implementar uma lógica crítica de pagamento em um e-commerce, a programação em par reduz o risco de erros financeiros que poderiam passar despercebidos em uma revisão de código posterior.

4. Liste e explique as fases do ciclo de vida de um projeto de software. Dê um exemplo prático para cada fase.

- **Concepção / Levantamento de Requisitos:** identificar o problema, definir o escopo e coletar os requisitos do sistema. Exemplo: entrevistar os gestores de uma clínica para entender o que o sistema de agendamento deve fazer.
- **Análise e Design:** modelar a arquitetura do sistema, criar diagramas de fluxo, definir banco de dados e interfaces. Exemplo: criar o diagrama ER do banco de dados e os wireframes das telas do sistema de agendamento.
- **Desenvolvimento:** codificar o sistema com base nos requisitos e no design aprovados. Exemplo: implementar o módulo de cadastro de pacientes e a lógica de agendamento de consultas.
- **Testes:** validar se o sistema funciona corretamente e atende aos requisitos. Exemplo: testar se dois pacientes não conseguem marcar o mesmo horário com o mesmo médico.
- **Implantação:** disponibilizar o sistema para os usuários finais em ambiente de produção. Exemplo: realizar o deploy do sistema na nuvem e treinar a equipe da clínica para utilizá-lo.
- **Manutenção:** corrigir bugs, implementar melhorias e adaptar o sistema a novas necessidades após o lançamento. Exemplo: adicionar a funcionalidade de teleconsulta após uma solicitação dos médicos.

5. Crie um quadro Kanban simples com três colunas: 'A Fazer', 'Em Progresso' e 'Concluído'. Adicione pelo menos cinco tarefas para um projeto de desenvolvimento de um aplicativo móvel.

Quadro Kanban - Projeto: Aplicativo Móvel de Controle Financeiro

A Fazer:

- Criar tela de cadastro e login do usuário.
- Desenvolver módulo de categorização de despesas.
- Integrar notificações push para alertas de limite de gastos.

Em Progresso:

- Implementar dashboard com gráfico de receitas e despesas mensais.
- Configurar autenticação com Google e Apple (OAuth 2.0).

Concluído:

- Definir arquitetura do projeto e configurar repositório no GitHub.
- Criar o design das telas principais no Figma (aprovado pelo cliente).

6. Descreva como a integração contínua pode ser usada para melhorar o fluxo de trabalho em um projeto ágil. Dê um exemplo prático.

A Integração Contínua (CI - Continuous Integration) é a prática de integrar o código de todos os desenvolvedores em um repositório compartilhado com frequência (várias vezes ao dia), com execução automática de testes a cada integração. Em projetos ágeis, ela melhora o fluxo de trabalho porque:

- Detecta conflitos e erros rapidamente: problemas de integração são identificados logo após o commit, quando ainda são fáceis de corrigir.
- Mantém o código sempre em estado funcional: a equipe pode realizar demos para o cliente a qualquer momento, sem depender de um ciclo longo de integração manual.
- Reduz o retrabalho: bugs encontrados cedo custam muito menos para corrigir do que bugs encontrados em produção.

Exemplo prático: uma equipe que usa GitHub Actions configura um pipeline que, a cada pull request, executa automaticamente os testes unitários e de integração. Se algum teste falhar, o merge é bloqueado até que o problema seja resolvido, garantindo que código quebrado nunca entre na branch principal.

7. Explique como os testes automatizados contribuem para o controle de qualidade no desenvolvimento de software.

Testes automatizados são scripts que verificam automaticamente se o software funciona conforme esperado, sem intervenção humana a cada execução. Sua contribuição para a qualidade inclui:

- Detecção rápida de regressões: ao modificar uma funcionalidade, os testes automatizados verificam instantaneamente se outras partes do sistema foram afetadas.
- Cobertura consistente: testes manuais tendem a ser seletivos e imprecisos; os automatizados executam sempre os mesmos cenários, garantindo consistência.
- Documentação viva: testes bem escritos descrevem o comportamento esperado do sistema, servindo como documentação técnica atualizada.
- Confiança para refatorar: com boa cobertura de testes, a equipe pode melhorar e refatorar o código sem medo de introduzir bugs não detectados.

8. Simule a organização de uma sprint no SCRUM para o desenvolvimento de um módulo de login de um sistema. Defina o objetivo da sprint, as tarefas e o tempo estimado.

Sprint 1 - Módulo de Login do Sistema

Objetivo da Sprint: implementar o módulo de autenticação completo, permitindo que usuários se cadastrem, façam login e recuperem senha com segurança.

Duração: 2 semanas (10 dias úteis)

Backlog da Sprint:

- Criar tela de cadastro de novo usuário (nome, e-mail, senha) - 8 horas.
- Implementar validação de formulário (campo obrigatório, formato de e-mail, força da senha) - 4 horas.
- Desenvolver endpoint de autenticação com JWT (JSON Web Token) - 10 horas.
- Criar tela de login com tratamento de erros (usuário ou senha incorretos) - 6 horas.

- Implementar fluxo de recuperação de senha via e-mail - 8 horas.
- Escrever testes automatizados para os endpoints de autenticação - 6 horas.

Total estimado: 42 horas. Definition of Done: todas as tarefas implementadas, testadas e aprovadas pelo Product Owner em ambiente de homologação.

9. Pesquise uma ferramenta de integração contínua e descreva como ela pode ser implementada em um projeto de software.

Ferramenta escolhida: GitHub Actions.

O GitHub Actions é uma plataforma de CI/CD integrada ao GitHub que permite automatizar pipelines de build, teste e deploy diretamente no repositório do projeto.

Como implementá-la:

- Criar um arquivo de workflow (.yml) na pasta .github/workflows do repositório, definindo os eventos que disparam o pipeline (ex.: push na branch main ou abertura de pull request).
- Configurar os jobs: instalar dependências, executar testes automatizados, verificar cobertura de código e, se tudo passar, fazer o deploy automaticamente para o ambiente de homologação ou produção.
- Configurar notificações: integrar com Slack ou e-mail para alertar a equipe em caso de falha no pipeline.

Benefício principal: o processo inteiro de validação e entrega é automatizado, eliminando etapas manuais sujeitas a erro e acelerando o ciclo de desenvolvimento.

10. Explique como o ciclo de vida de um projeto de software pode ser adaptado ao uso de metodologias ágeis, como SCRUM e XP.

No modelo tradicional (Waterfall), as fases do ciclo de vida são sequenciais e rígidas. Com metodologias ágeis, essas fases se tornam iterativas e sobrepostas:

- Levantamento de requisitos: em vez de coletar todos os requisitos de uma vez, o SCRUM faz isso de forma contínua. O backlog é refinado a cada Sprint com base no feedback dos usuários.
- Design e desenvolvimento: no XP, o design é feito de forma simples e evoluído conforme necessário (design emergente), evitando arquiteturas excessivamente complexas criadas antes de entender o problema real.
- Testes: integrados ao ciclo de desenvolvimento. No XP, os testes são escritos antes do código (TDD - Test-Driven Development), tornando-se parte natural de cada entrega.
- Implantação: no SCRUM, ao final de cada Sprint há um incremento potencialmente entregável. Com CD (Continuous Delivery), deploys podem ocorrer a qualquer momento de forma automatizada.

O resultado é um ciclo de vida mais curto, adaptável e orientado a valor, onde o produto evolui com base em aprendizado real em vez de suposições iniciais.